K-Means Image Segmentation - Project Report

1. Project Objective

This project implements K-means clustering for image segmentation using multiple parallelization approaches. The goal is to segment images by grouping similar-colored pixels into clusters, effectively reducing the color palette while preserving the overall visual structure of the image.

Here we will:

- Take an input image and reduce its color complexity by grouping similar colors
- Implement the K-means algorithm in multiple ways to compare performance
- Explore different parallelization strategies: single-threaded CPU, multi-threaded CPU (OpenMP), and GPU acceleration (CUDA)
- Optimize GPU performance using shared memory techniques
- Benchmark and compare the performance of all implementations

About K-means Algorithm:

The algorithm works by:

1. Randomly initializing K cluster centroids (representative colors)
2. Assigning each pixel to its nearest centroid (based on color distance)
3. Updating centroids to be the mean of all pixels assigned to them
4. Repeating steps 2-3 until convergence (centroids stop moving significantly)

This process effectively groups pixels with similar colors together, creating a segmented image with K distinct color regions.

---

2. Implementation Versions

2.1 CPU Single-Threaded Version: `kmeans_cpu.cpp`

- Sequential processing of all pixels
- Each pixel is processed one at a time in nested loops

- No parallelization - uses a single CPU core
- Baseline implementation for performance comparison

2.2 OpenMP Multi-Threaded Version: `kmeans_omp.cpp`

- Parallelizes the computationally intensive loops using OpenMP
- Divides work across multiple CPU cores
- Uses thread-local storage to avoid race conditions
- Leverages OpenMP reduction for error computation
- Parallelized with `#pragma omp parallel for`
- Update step: Uses thread-local accumulators, then combines results

Code structure: - : Same algorithm as CPU, but with OpenMP pragmas - Thread-safe accumulation using per-thread arrays - Automatic thread management by OpenMP

Performance: - Typically 5-6x speedup over single-threaded CPU - Scales with number of CPU cores - Good balance between performance and code complexity

---

2.3 CUDA GPU Version (Global Memory) `kmeans_cuda.cu`

- Offloads computation to GPU for massive parallelism
- Uses one thread per pixel for parallel processing
- All data stored in GPU global memory
- Requires CPU-GPU memory transfers
- Each GPU thread processes one pixel
- Uses atomic operations to accumulate centroids
- **Memory transfers**: Data copied to/from GPU before/after computation
- Device functions for color space conversion
- Kernel launch configuration with 256 threads per block

Performance: - May be slower on small images due to GPU overhead - Better for larger images where parallelism pays off - First run includes GPU initialization overhead

Limitations: - All centroid lookups access global memory (slower) - Memory transfer overhead between CPU and GPU - Requires CUDA-capable GPU

2.4 CUDA GPU Version (Shared Memory): `kmeans_cuda_shared.cu`

- Same GPU parallelism as global memory version
- Loads centroids into fast shared memory
- Uses cooperative loading pattern (threads work together)
- Reduces global memory accesses significantly
- **Shared memory for centroids**: Centroids loaded once per block into shared memory
- **Cooperative loading**: Threads distribute the work of loading centroids
- **16x16 thread blocks**: Optimized block size (256 threads) like the reference implementation
- **Synchronization**: `__syncthreads()` ensures data is loaded before use
- Uses `extern __shared__` for dynamic shared memory allocation
- Same kernel structure but with shared memory access pattern

Shared memory pattern (from reference implementation):

```
extern __shared__ float sharedCentroids[];
int tid = threadIdx.y * blockDim.x + threadIdx.x;
int blockSize = blockDim.x * blockDim.y;

// Cooperative loading
for (int i = tid; i < totalFloats; i += blockSize) {
    sharedCentroids[i] = globalCentroids[i];
}
__syncthreads();  // Wait for all threads to finish loading
```

3. Code Details and Results

3.1 Code Architecture

Project Structure:

```
.
├── include/
│   ├── kmeans_cpu.h          # CPU implementation interface
│   ├── kmeans_omp.h          # OpenMP implementation interface
│   ├── kmeans_cuda.h         # CUDA global memory interface
│   ├── kmeans_cuda_shared.h  # CUDA shared memory interface
│   ├── image_loader.h        # Image I/O utilities
│   └── timer.h               # Performance timing
├── src/
│   ├── main.cpp              # Main program with CLI
│   ├── kmeans_cpu.cpp        # CPU implementation
│   ├── kmeans_omp.cpp        # OpenMP implementation
│   ├── kmeans_cuda.cu        # CUDA global memory
│   ├── kmeans_cuda_shared.cu # CUDA shared memory
│   ├── image_loader.cpp      # Image loading/saving
│   └── timer.cpp             # Timer implementation
└── lib/stb/                  # stb_image library
```

- All versions share the same `KMeansResult` structure for consistency
- Color space conversion (RGB ↔ Lab) implemented in each version
- Random centroid initialization from actual image pixels
- Convergence detection based on centroid movement threshold

3.2 Results from Each Method

All four implementations were tested on `scene.jpg` (4672x7008 pixels) with k=20 and RGB color space:

Result Images:

1. **CPU Single-Threaded** (`output_scene_cpu.png`)
2. Baseline result
3. Produces the reference segmentation
4. All other versions should match this (within numerical precision)
5. **OpenMP Multi-Threaded** (`output_scene_omp.png`)
6. Same visual result as CPU
7. Faster execution time
8. Demonstrates multi-core CPU utilization

9. **CUDA GPU - Global Memory** (`output_scene_gpu.png`)
10. Same visual result
11. GPU-accelerated computation
12. Shows GPU parallel processing capability
13. **CUDA GPU - Shared Memory** (`output_scene_gpu_shared.png`)
14. Same visual result
15. Optimized GPU implementation
16. Note: Performance issues observed - requires investigation

Result Images:

CPU Single-Threaded Segmentation Result

OpenMP Multi-Threaded Segmentation Result

CUDA GPU Global Memory Segmentation Result

CUDA GPU Shared Memory Segmentation Result

Visual Analysis: All four output images should appear visually identical, demonstrating that: - All implementations produce the same segmentation result - The algorithm is correctly implemented across all versions - Only performance differs, not the output quality

Performance Comparison:

Individual Run Results (scene.jpg, 4672x7008 pixels, k=20, RGB):

- **CPU Single-Threaded**:

- Runtime: 132561.57 ms (132.56 seconds)
- Iterations: 100
- Final error: 4.39602e+09
- Total time: 133020.02 ms
- Centroids:
  - RGB(43,14,9), RGB(78,72,72), RGB(61,53,49), RGB(159,45,16), RGB(107,107,108)
  - RGB(11,4,2), RGB(219,113,60), RGB(85,24,10), RGB(3,1,0), RGB(126,126,123)
  - RGB(183,162,125), RGB(24,6,3), RGB(200,187,146), RGB(90,90,94), RGB(151,151,141)
  - RGB(180,186,165), RGB(42,36,34), RGB(127,32,10), RGB(221,196,165), RGB(21,20,19)
- **OpenMP Multi-Threaded** (4 threads):
- Runtime: 39236.43 ms (39.24 seconds)
- Iterations: 100
- Final error: 4.21741e+09
- Total time: 39410.85 ms
- **Speedup: 3.38x** over CPU
- **Improvement: 70.4% faster**
- Centroids:
  - RGB(57,48,44), RGB(88,88,93), RGB(196,190,163), RGB(171,182,170), RGB(4,2,0)
  - RGB(75,67,65), RGB(48,16,9), RGB(217,111,59), RGB(232,200,165), RGB(107,106,107)
  - RGB(152,41,14), RGB(107,27,10), RGB(126,126,123), RGB(11,4,1), RGB(197,172,131)
  - RGB(37,33,31), RGB(19,18,17), RGB(156,151,136), RGB(25,7,3), RGB(1,1,0)
- **CUDA GPU (Global Memory)**:
- Runtime: 19100.43 ms (19.10 seconds)
- Iterations: 89 (converged early)
- Final error: 4.0082e+09
- Total time: 19101.94 ms
- **Speedup: 6.94x** over CPU
- **Speedup: 2.05x** over OpenMP

- Note: GPU demonstrates superior performance on large images
- Centroids:
  - RGB(160,161,151), RGB(20,8,6), RGB(189,193,169), RGB(214,108,57), RGB(105,105,106)
  - RGB(205,181,136), RGB(136,35,11), RGB(170,185,173), RGB(189,165,125), RGB(166,150,122)
  - RGB(238,203,167), RGB(140,141,134), RGB(50,42,38), RGB(122,123,120), RGB(87,86,91)
  - RGB(4,2,0), RGB(37,21,17), RGB(211,192,160), RGB(191,184,157), RGB(71,62,58)
- **CUDA GPU (Shared Memory)**:
- Runtime: 484804.23 ms (484.80 seconds / 8.08 minutes)
- Iterations: 92 (converged early)
- Final error: 3.98024e+09
- Total time: 484805.69 ms
- Note: **Severe performance degradation** - significantly slower than all other versions, requires optimization
- Centroids:
  - RGB(165,174,166), RGB(107,107,108), RGB(126,126,123), RGB(197,172,131), RGB(89,89,94)
  - RGB(107,28,10), RGB(233,200,166), RGB(22,20,19), RGB(61,52,48), RGB(181,189,171)
  - RGB(198,190,161), RGB(77,71,70), RGB(25,7,3), RGB(11,4,2), RGB(155,150,134)
  - RGB(152,41,14), RGB(41,36,33), RGB(3,1,0), RGB(49,16,9), RGB(217,111,59)

Performance Analysis:

Key Observations:

- **CUDA Global Memory is fastest** for large images (4672x7008 pixels) with 6.94x speedup over CPU and 2.05x speedup over OpenMP
- **OpenMP provides significant speedup** (3.38x) over single-threaded CPU, demonstrating effective multi-core utilization
- **GPU performance scales well** with image size - the large image (32.7 megapixels) fully utilizes GPU parallelism

- **Shared memory version shows severe performance issues** - 25.4x slower than global memory version, indicating potential synchronization or memory access pattern problems

Performance Summary:

- **CPU Single-Threaded**: 132.56 seconds (baseline)
- **OpenMP Multi-Threaded**: 39.24 seconds (3.38x faster than CPU)
- **CUDA Global Memory**: 19.10 seconds (6.94x faster than CPU, 2.05x faster than OpenMP)
- **CUDA Shared Memory**: 484.80 seconds (3.66x slower than CPU)

Why CUDA excels on large images:

- Large workload (32.7 megapixels) fully utilizes GPU parallelism
- Thousands of threads can process pixels simultaneously
- Memory transfer overhead becomes negligible compared to computation time
- GPU's parallel architecture is ideal for pixel-level operations

4. Instructions

4.1 Prerequisites

- C++17 compiler (GCC, Clang, or MSVC)
- CMake 3.15+
- OpenMP (required for multi-threaded version)
- CUDA Toolkit (optional, for GPU versions)
- OpenCV (optional, for image I/O) OR stb_image (included)

4.2 Building the Project

Step 1: Load required modules

```
module load cmake
module load cudatoolkit  # If using CUDA
```

Step 2: Set library path (if needed)

```
export LD_LIBRARY_PATH="/usr/lib64/nagios/plugins/python3/lib:$LD_LIBRARY_PATH"
```

Step 3: Build

```
cd /project/hnguyen2/mvu9/folder_04_ma/final_project
make clean
make build
```

Or manually:

```
mkdir build
cd build
cmake ..
make
```

4.3 Usage

Basic syntax:

```
./build/kmeans_segmentation <image_path> <k> <color_space> [options]
```

Parameters: - `image_path`: Path to input image (PNG, JPG, etc.) - `k`: Number of clusters (3-100) - `color_space`: `rgb` or `lab`

Options: - `--version <cpu|omp|cuda|cuda_shared|benchmark>`: Choose implementation - `--output <path>`: Output file path (default: `output_segmented.png`) - `--threads <n>`: Number of OpenMP threads (default: auto)

4.4 Running All Versions

Quick method - use the script:

```
./run_all_versions_scene.sh
```

Or:

```
make run-all
```

Using Makefile targets:

```
make run-cpu    # Creates output_scene_cpu.png
make run-omp    # Creates output_scene_omp.png
make run-cuda   # Creates output_scene_gpu.png
```

Individual commands:

```
# CPU version
./build/kmeans_segmentation scene.jpg 5 rgb --version cpu --output output_scene

# OpenMP version
./build/kmeans_segmentation scene.jpg 5 rgb --version omp --threads 4 --output

# CUDA GPU version (global memory)
./build/kmeans_segmentation scene.jpg 5 rgb --version cuda --output output_scen

# CUDA GPU version (shared memory - optimized)
./build/kmeans_segmentation scene.jpg 5 rgb --version cuda_shared --output outp
```

4.5 Benchmarking

To compare all versions with performance metrics:

```
./build/kmeans_segmentation scene.jpg 5 rgb --version benchmark
```

This will: - Run each version multiple times (3 runs) - Show min, max, and average execution times - Calculate speedup factors - Verify that results match between versions - Display detailed performance statistics

4.6 Examples

Different color spaces:

```
# RGB color space
./build/kmeans_segmentation scene.jpg 5 rgb --version omp

# Lab color space (better perceptual clustering)
./build/kmeans_segmentation scene.jpg 5 lab --version cuda_shared
```

Different number of clusters:

```
# 3 clusters (simpler segmentation)
./build/kmeans_segmentation scene.jpg 3 rgb --version cuda_shared

# 8 clusters (more detailed segmentation)
./build/kmeans_segmentation scene.jpg 8 rgb --version cuda_shared
```

Custom output path:

```
./build/kmeans_segmentation scene.jpg 5 rgb --version cuda_shared --output my_re
```

4.7 Troubleshooting

CMake not found:

```
module load cmake
```

CUDA not found:

```
module load cudatoolkit
which nvcc  # Verify CUDA is loaded
```

Library errors: (if you

```
export LD_LIBRARY_PATH="/usr/lib64/nagios/plugins/python3/lib:$LD_LIBRARY_PATH"
```

Build errors: - Make sure all modules are loaded - Try `make clean` then `make build` - Check that CUDA is available if building GPU versions

4.8 Expected Output Files

After running all versions, you should have: - `output_scene_cpu.png` - CPU single-threaded result - `output_scene_omp.png` - OpenMP multi-threaded result - `output_scene_gpu.png` - CUDA global memory result - `output_scene_gpu_shared.png` - CUDA shared memory result

All images should be visually identical, showing the same segmentation with different performance characteristics.

Example: Running all versions on scene.jpg (4672x7008 pixels, k=20):

```
$ ./run_all_versions_scene.sh
running all versions of k-means segmentation...
input image: scene.jpg
k=20, color space=rgb

=== running cpu single-threaded version ===
Loading image: scene.jpg
Image loaded: 4672x7008 (3 channels)
Running K-means with k=20, color space=rgb

=== Results ===
Iterations: 100
Final error: 4.39602e+09
Runtime (CPU single-thread): 132561.57 ms
Total time: 133020.02 ms

Saving segmented image to: output_scene_cpu.png
Successfully saved segmented image!

Final centroids:
  Cluster 0: RGB(43, 14, 9)
  Cluster 1: RGB(78, 72, 72)
  Cluster 2: RGB(61, 53, 49)
  Cluster 3: RGB(159, 45, 16)
  Cluster 4: RGB(107, 107, 108)
  Cluster 5: RGB(11, 4, 2)
  Cluster 6: RGB(219, 113, 60)
  Cluster 7: RGB(85, 24, 10)
  Cluster 8: RGB(3, 1, 0)
  Cluster 9: RGB(126, 126, 123)
  Cluster 10: RGB(183, 162, 125)
  Cluster 11: RGB(24, 6, 3)
  Cluster 12: RGB(200, 187, 146)
  Cluster 13: RGB(90, 90, 94)
  Cluster 14: RGB(151, 151, 141)
  Cluster 15: RGB(180, 186, 165)
  Cluster 16: RGB(42, 36, 34)
  Cluster 17: RGB(127, 32, 10)
  Cluster 18: RGB(221, 196, 165)
  Cluster 19: RGB(21, 20, 19)
```

```
=== running openmp multi-threaded version ===
Loading image: scene.jpg
Image loaded: 4672x7008 (3 channels)
Running K-means with k=20, color space=rgb
Using OpenMP with 4 thread(s)

=== Results ===
Iterations: 100
Final error: 4.21741e+09
Runtime (OpenMP multi-thread): 39236.43 ms
Total time: 39410.85 ms

Saving segmented image to: output_scene_omp.png
Successfully saved segmented image!

Final centroids:
  Cluster 0: RGB(57, 48, 44)
  Cluster 1: RGB(88, 88, 93)
  Cluster 2: RGB(196, 190, 163)
  Cluster 3: RGB(171, 182, 170)
  Cluster 4: RGB(4, 2, 0)
  Cluster 5: RGB(75, 67, 65)
  Cluster 6: RGB(48, 16, 9)
  Cluster 7: RGB(217, 111, 59)
  Cluster 8: RGB(232, 200, 165)
  Cluster 9: RGB(107, 106, 107)
  Cluster 10: RGB(152, 41, 14)
  Cluster 11: RGB(107, 27, 10)
  Cluster 12: RGB(126, 126, 123)
  Cluster 13: RGB(11, 4, 1)
  Cluster 14: RGB(197, 172, 131)
  Cluster 15: RGB(37, 33, 31)
  Cluster 16: RGB(19, 18, 17)
  Cluster 17: RGB(156, 151, 136)
  Cluster 18: RGB(25, 7, 3)
  Cluster 19: RGB(1, 1, 0)

=== running cuda gpu version (global memory) ===
Loading image: scene.jpg
Image loaded: 4672x7008 (3 channels)
```

```
Running K-means with k=20, color space=rgb
Converged at iteration 89

=== Results ===
Iterations: 89
Final error: 4.0082e+09
Runtime (CUDA GPU - Global Memory): 19100.43 ms
Total time: 19101.94 ms

Saving segmented image to: output_scene_gpu.png
Successfully saved segmented image!

Final centroids:
  Cluster 0: RGB(160, 161, 151)
  Cluster 1: RGB(20, 8, 6)
  Cluster 2: RGB(189, 193, 169)
  Cluster 3: RGB(214, 108, 57)
  Cluster 4: RGB(105, 105, 106)
  Cluster 5: RGB(205, 181, 136)
  Cluster 6: RGB(136, 35, 11)
  Cluster 7: RGB(170, 185, 173)
  Cluster 8: RGB(189, 165, 125)
  Cluster 9: RGB(166, 150, 122)
  Cluster 10: RGB(238, 203, 167)
  Cluster 11: RGB(140, 141, 134)
  Cluster 12: RGB(50, 42, 38)
  Cluster 13: RGB(122, 123, 120)
  Cluster 14: RGB(87, 86, 91)
  Cluster 15: RGB(4, 2, 0)
  Cluster 16: RGB(37, 21, 17)
  Cluster 17: RGB(211, 192, 160)
  Cluster 18: RGB(191, 184, 157)
  Cluster 19: RGB(71, 62, 58)

=== running cuda gpu version (shared memory) ===
Loading image: scene.jpg
Image loaded: 4672x7008 (3 channels)
Running K-means with k=20, color space=rgb
Converged at iteration 92

=== Results ===
```

```
Iterations: 92
Final error: 3.98024e+09
Runtime (CUDA GPU - Shared Memory): 484804.23 ms
Total time: 484805.69 ms

Saving segmented image to: output_scene_gpu_shared.png
Successfully saved segmented image!

Final centroids:
  Cluster 0: RGB(165, 174, 166)
  Cluster 1: RGB(107, 107, 108)
  Cluster 2: RGB(126, 126, 123)
  Cluster 3: RGB(197, 172, 131)
  Cluster 4: RGB(89, 89, 94)
  Cluster 5: RGB(107, 28, 10)
  Cluster 6: RGB(233, 200, 166)
  Cluster 7: RGB(22, 20, 19)
  Cluster 8: RGB(61, 52, 48)
  Cluster 9: RGB(181, 189, 171)
  Cluster 10: RGB(198, 190, 161)
  Cluster 11: RGB(77, 71, 70)
  Cluster 12: RGB(25, 7, 3)
  Cluster 13: RGB(11, 4, 2)
  Cluster 14: RGB(155, 150, 134)
  Cluster 15: RGB(152, 41, 14)
  Cluster 16: RGB(41, 36, 33)
  Cluster 17: RGB(3, 1, 0)
  Cluster 18: RGB(49, 16, 9)
  Cluster 19: RGB(217, 111, 59)

=== all versions completed! ===
output files:
  - output_scene_cpu.png (cpu single-threaded)
  - output_scene_omp.png (openmp multi-threaded)
  - output_scene_gpu.png (cuda gpu - global memory)
 - output_scene_gpu_shared.png (cuda gpu - shared memory)
```